DATABASE MANAGEMENT SYSTEM (CT 652)

Tribhuvan University | Institute of Engineering | BCT | Year III / Part II

CHAPTER 6: Storage, File Structure and Indexing

Syllabus Hours: 8 hrs | Marks Weight: 14 marks

Complete Exam-Ready Answers | All Past Questions Covered (2070-2082)

CHAPTER 6 SYLLABUS TOPICS

- **6.1** Overview of Physical Storage Media
- **6.2** Magnetic Disk and Flash Storage
- **6.3** RAID - Redundant Array of Independent Disks
- **6.4** Tertiary Storage
- **6.5** Storage Access - Buffer Manager
- **6.6** File Organization
- **6.7** Organization of Records in Files
- **6.8** Data Dictionary Storage
- **6.9** Basic Concepts of Indexing
- **6.10** Ordered Indices - Dense and Sparse Index
- **6.11** B+ Tree Index Files
- **6.12** B-Tree Index Files
- **6.13** Static Hashing
- **6.14** Dynamic Hashing - Extendible Hashing
- **6.15** Query Processing and Optimization Overview

SECTION 1: Physical Storage Media and Magnetic Disk

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2070 Magh

Q1. What is physical storage media? Classify different types of storage media used in DBMS based on speed, cost, and volatility. Draw the storage hierarchy.	**[2+6]** *2081 Chaitra 2079 Chaitra 2075 Bhadra*

Answer: Physical Storage Media Classification

Physical storage media are the hardware devices used to store data in a DBMS. Media differ in speed, cost per GB, capacity, and volatility (whether data persists without power). The DBMS manages a hierarchy of these media to balance performance and cost.

Storage Hierarchy (Fastest/Costliest to Slowest/Cheapest):

Level	Storage Type	Access Speed	Cost/GB	Volatile?	Typical Capacity	Example
1	Cache Memory	~1 ns	Very High	YES	KB-MB	CPU L1/L2/L3
2	Main Memory (RAM)	~10-100 ns	High	YES	GB	DDR4/DDR5 RAM
3	Flash / SSD	~10-100 us	Medium	NO	GB-TB	NVMe SSD
4	Magnetic Disk (HDD)	~5-10 ms	Low	NO	TB	Hard disk drive
5	Optical Disk	~100-500 ms	Very Low	NO	GB-TB	Blu-ray, DVD
6	Magnetic Tape	Seconds	Cheapest	NO	TB-PB	LTO tape

Category	Storage Types	Key Property
Primary Storage (Volatile)	Cache, Main Memory (RAM)	Directly CPU-accessible. Fastest. Data lost on power failure. Holds active data during processing.
Secondary Storage (Non-Volatile)	Flash/SSD, Magnetic Disk (HDD)	Persistent - data survives power loss. Main DBMS data storage. Block-based I/O.
Tertiary Storage (Non-Volatile)	Optical disks, Magnetic Tape	Offline/near-line archival. Extremely cheap per GB. Used for backups and historical archives.

Q2. Explain the structure of a Magnetic Disk. Define: platter, track, sector, cylinder, seek time, rotational latency, and transfer rate. How is disk access time calculated?

[2+6]
2081 Ashwin 2079
Chaitra 2076 Baisakh

Answer: Magnetic Disk Structure and Access Time

A magnetic disk (HDD) stores data on rotating platters coated with magnetic material. A movable read/write arm accesses data by positioning heads over the correct track and waiting for the desired sector to rotate underneath.

Key Components of a Magnetic Disk:

Component	Definition	Role in DBMS
Platter	Circular disk coated with magnetic material. Multiple platters (2-10) stacked on a common spindle.	Multiple platters increase total storage capacity of the disk.

Track	A concentric ring on a platter surface. Each platter surface has thousands of tracks.	Data is stored along tracks. Head reads/writes data as the track rotates under it.
Sector	The smallest unit of storage on a disk (512 bytes or 4KB). Each track is divided into sectors.	Minimum unit of disk I/O. DBMS groups sectors into larger blocks (pages).
Block (Page)	A contiguous set of sectors treated as a single I/O unit by DBMS. Typically 4KB-16KB.	The fundamental unit of data transfer between disk and memory buffer.
Cylinder	The set of all tracks at the same radial position across all platters (directly above/below each other).	Accessing data within the same cylinder avoids seek movement - much faster.
Disk Arm	The mechanical arm that moves all read/write heads together radially across the disk.	Seek movement: arm moves to position head over the correct track.
Read/Write Head	Electromagnetic sensor at tip of disk arm. One head per platter surface.	Reads and writes magnetic data on the platter surface.

Disk Access Time Components:

Component	Definition	Typical Value	Optimization
Seek Time	Time for disk arm to MOVE head to correct TRACK. Dominant component of access time.	3-15 ms average	Store related data in same cylinder. Use indexes.
Rotational Latency	Time for disk to ROTATE until desired SECTOR is under head. Average = 1/2 rotation period.	~4 ms at 7200 RPM	Arrange data to minimize rotational wait.
Transfer Time	Time to READ or WRITE the actual block data once head is positioned.	~0.04 ms (tiny)	Use large block sizes to amortize seek/rotation cost.
Total Access Time	Seek Time + Rotational Latency + Transfer Time	~9-20 ms typical	Use buffer pool to cache hot blocks in RAM.

Magnetic Disk - Simplified Structure Diagram

```
+-----------------------------------------------+
|            MAGNETIC DISK (HDD)                |
|                                               |
|   Spindle (center rotation axis)              |
|        |                                      |
|  +----|---------------------------------------|
|  |Platter (top view):                         |
```

```
   |  |                                          |
   |  |  Track 3 (outermost) ............        |
   |  |    Track 2 ........                      |
   |  |      Track 1 ....  <- Sector (arc)       |
   |  +------------------------------------------+
   |                                             |
   |  ===Disk Arm==> [R/W Head]   moves radially  |
   |                                             |
   |  Cylinder = All Track 1s stacked vertically  |
   +---------------------------------------------+

   Access Time = Seek Time + Rotational Latency + Transfer Time
```

Note: Seek time dominates. Minimize it by: (1) using indexes to read fewer blocks, (2) placing related data in same cylinder, (3) using buffer pool to cache hot data in RAM.

Q3. What is RAID? Explain RAID levels 0, 1, 5, and 6 with diagrams. Compare their reliability and storage efficiency. Which RAID level is best for a DBMS?

[2+4+2]

2081 Chaitra 2079
Ashwin 2077 Chaitra

Answer: RAID - Redundant Array of Independent Disks

RAID: RAID uses multiple physical disks to provide improved performance, fault tolerance, or both - appearing as a single logical disk. RAID uses two key techniques: (1) Striping: distributing data across disks for parallel I/O performance. (2) Mirroring/Parity: storing redundant information for fault tolerance.

RAID Level	Name	Mechanism	Fault Tolerance	Storage Efficiency	Min Disks	Best For
RAID 0	Striping	Data striped in blocks across N disks. No redundancy.	NONE - any disk failure = data loss.	100% (no overhead)	2	High speed, non-critical data. Video editing.
RAID 1	Mirroring	Every write goes to primary disk AND mirror disk simultaneously.	YES - survives 1 disk failure.	50% (stores 2x)	2	Critical OS drives, database logs, boot disks.
RAID 5	Stripe+Parity	Data AND parity distributed across all N disks. Parity rotates.	YES - survives 1 disk failure. Rebuild from parity.	(N-1)/N	3	General NAS, web servers, DBMS data files.

RAID 6	Double Parity	Like RAID 5 but TWO parity blocks per stripe on different disks.	YES - survives 2 simultaneous disk failures.	(N-2)/N	4	Mission-critical, high reliability needed.
RAID 10	Stripe+Mirror	First mirror pairs (RAID 1), then stripe across pairs (RAID 0).	YES - 1 failure per mirrored pair.	50%	4	Highest performance + reliability: database servers.

RAID Levels - Visual Comparison

```
RAID 0 (Striping - 3 disks):
  Disk1: [A1][A4][A7]   Disk2: [A2][A5][A8]   Disk3: [A3][A6][A9]
  Speed: 3x  |  Reliability: 0  (no redundancy)

RAID 1 (Mirroring - 2 disks):
  Disk1: [A1][A2][A3]   Disk2: [A1][A2][A3]  (exact copy)
  Speed: 2x read  |  Reliability: 1 disk failure OK

RAID 5 (Distributed Parity - 3 disks):
  Disk1: [A1][A4][P3]   Disk2: [A2][P2][A5]   Disk3: [P1][A3][A6]
  P1=A1 XOR A2 XOR A3 (parity for stripe 1 stored on Disk3)
  Speed: (N-1)x  |  Reliability: 1 disk failure OK  |  Efficiency: 2/3

RAID 10 (Mirror then Stripe - 4 disks):
  Pair1: Disk1=[A1][A2] Mirror with Disk2=[A1][A2]
  Pair2: Disk3=[A3][A4] Mirror with Disk4=[A3][A4]
  Stripe across pairs: Speed high + Reliability high
```

Note: Best for DBMS: RAID 10 (maximum performance + reliability) or RAID 5 (cost-effective). RAID 5 is most common for DBMS data files; RAID 10 for transaction logs.

SECTION 2: Buffer Manager and File Organization

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin

Q4. What is the Buffer Manager in DBMS? Explain buffer replacement policies: LRU, MRU, and Clock algorithm. What is pin count and a dirty page?	**[2+4+2]** *2081 Ashwin 2079 Chaitra 2075 Bhadra*

Answer: Buffer Manager and Replacement Policies

Buffer Pool: The buffer pool is a region of main memory (RAM) managed by the DBMS Buffer Manager. It holds copies of disk blocks (pages) being used or recently used. Since disk access is ~10,000x slower than RAM, caching hot pages in the buffer pool dramatically reduces I/O.

Key Concepts:

Concept	Definition
Buffer Frame	One slot in the buffer pool that can hold one disk block (page). Total frames = buffer pool size / page size.
Pin Count	A counter per buffer frame. Incremented when a transaction starts using the page (pin), decremented when done (unpin). A page with pin count > 0 CANNOT be evicted.
Dirty Page	A page that has been modified in the buffer but not yet flushed (written back) to disk. Must be written to disk before eviction (else changes are lost).
Buffer Hit	The requested page is already in the buffer pool - no disk I/O needed. Hit rate = hits / total requests. Higher is better.
Buffer Miss	The requested page is NOT in the buffer pool - must load from disk. If no free frame, a victim page must be evicted first.

Buffer Replacement Policies:

Policy	Algorithm	When Best	Limitation
LRU (Least Recently Used)	Evict the page whose LAST ACCESS was the LONGEST time ago. Tracks access timestamp for each frame.	General workloads with temporal locality - recently used pages likely needed again.	Poor for sequential scans (every page accessed once; all equally recent).

MRU (Most Recently Used)	Evict the page that was MOST RECENTLY accessed.	Sequential full table scans - most recently used page will NOT be needed again soon.	Counter-intuitive for random access workloads.
Clock (Second-Chance)	Circular list of frames, each with a reference bit. On access: set bit=1. On replacement scan: if bit=1, set to 0 and skip (second chance). If bit=0, evict this page.	Efficient approximation of LRU. Low overhead - no timestamp needed.	Slightly less accurate than true LRU.
FIFO	Evict the page that has been in the buffer the LONGEST (first loaded).	Simple to implement.	Ignores actual usage frequency - old but hot pages may be wrongly evicted.

Q5. Explain the different types of File Organization in DBMS. Compare heap, sequential, hash, and clustered file organizations. What is the difference between fixed-length and variable-length records?

[2+4+2]

2080 Ashwin 2078
Chaitra 2074 Bhadra

Answer: File Organization Types

File Organization: File organization defines how records are physically stored on disk. It determines the efficiency of insert, delete, and query operations. Different organizations suit different workloads.

Organization	How Records Stored	Insert	Exact Search	Range Search	Best For
Heap File (Unordered)	Records appended in any order - typically at end of file. No ordering maintained.	Very Fast O(1)	Full scan O(n)	Full scan O(n)	Write-heavy workloads; bulk loading; no frequent queries.
Sequential File (Sorted)	Records ordered physically by a sort key (usually primary key). New records may require shifting.	Slow O(n) (insertion sort)	Binary search O(log n)	Fast O(log n)	Read-heavy; range queries; ordered reports; static data.
Hash File	Records placed in bucket h(key). Direct access by key value.	O(1) average	O(1) average (hash)	NOT supported	Exact-match point queries on primary key.
Clustered File	Records from related tables stored together in same block (interleaved rows).	Complex	Depends on index	Depends on index	Frequently joined tables (e.g., Employee + Department).

Fixed-Length vs Variable-Length Records:

Feature	Fixed-Length Records	Variable-Length Records
Definition	Every record has exactly the same size in bytes.	Records differ in size due to VARCHAR fields, NULLs, or optional attributes.
Record Location	Calculated arithmetically: record i at offset i x record_size.	Requires a directory/slot array to locate each record - offsets stored in page header.
Wasted Space	May waste space if actual data is smaller than max size.	No wasted space for NULL/short values. But needs extra management overhead.
Implementation	Simple - direct byte offset calculation.	Complex - slotted page structure, null bitmaps, variable-length field descriptors.
Example	Employee(empid INT, salary FLOAT) = always 8 bytes.	Employee(empid INT, name VARCHAR(100)) = 4 to 104 bytes per record.

SECTION 3: Indexing - Basic Concepts, Dense and Sparse Indices

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q6. What is indexing in DBMS? Why is it needed? Define: search key, primary index, secondary index, dense index, and sparse index. What are the trade-offs?

[2+6]
2081 Chaitra 2079 Chaitra 2075 Bhadra

Answer: Indexing - Basic Concepts

Index: An index is an auxiliary data structure that enables fast access to records based on the value of certain attributes (the search key). Without an index, finding a record requires scanning the entire file O(n). With an index, access is O(log n) or O(1). An index consists of entries: (search-key value, pointer to record or block).

Why Indexing is Needed:

* Databases may have millions of records - sequential scan is too slow for user queries.
* Indexes allow the DBMS to jump directly to records matching a search condition.
* Multiple indexes on different attributes enable fast access for different query types.
* Trade-off: indexes speed up reads but slow down writes (index must be updated on INSERT/UPDATE/DELETE).

Index Terminology:

Term	Definition	Example
Search Key	The attribute(s) on which the index is built. Used to look up records.	salary in an index on Employee.salary.
Primary Index (Clustering)	Index built on the search key that determines PHYSICAL ORDER of records. File sorted on this key. Only one primary index per file.	Index on empid when file is sorted by empid.
Secondary Index (Non-Clustering)	Index on an attribute that does NOT determine physical record order. Multiple secondary indexes allowed.	Index on salary when file is sorted by empid.
Dense Index	One index entry for EVERY record in the data file.	1000 records = 1000 index entries.
Sparse Index	One index entry for only SOME records - typically one entry per DISK BLOCK. Requires sorted (clustered) file.	100 disk blocks = 100 index entries.
Multilevel Index	An index on the index - used when even the index itself is too large for memory.	Outer sparse index on inner dense index. Basis for B+ tree.

Q7. Compare Dense Index and Sparse Index with diagrams showing their structure. When would you prefer each? What is a Multilevel Index and why is it needed?

[2+4+2]
2080 Ashwin 2078 Chaitra 2073 Bhadra

Answer: Dense vs Sparse Index

Feature	Dense Index	Sparse Index
Entries Count	One entry per RECORD in data file. Largest index size.	One entry per BLOCK (typically). Much smaller index.
Applicable On	Any file (sorted or unsorted).	ONLY on clustered (physically sorted) files.
Search Speed	Fast - directly locate record via pointer.	Slightly slower - locate block, then scan within block.
Index Size	Large (equals number of records).	Small (equals number of disk blocks).
Update Overhead	High - every insert/delete updates the index.	Low - only update when a new block is created.
Pointer Type	Points to individual RECORD.	Points to first record in the BLOCK.

Dense Index vs Sparse Index - Visual Structure

```
DENSE INDEX (one entry per record):
  Index Entries           DATA FILE (any order OK)
  +-----------+         +---------------------------+
  | 10 | ptr1 |-------->| empid=10, name=Ram, sal=50K|
  | 20 | ptr2 |-------->| empid=20, name=Sita,sal=60K|
  | 30 | ptr3 |-------->| empid=30, name=Hari,sal=45K|
  | 40 | ptr4 |-------->| empid=40, name=Gita,sal=70K|
  +-----------+         +---------------------------+
  4 records = 4 index entries

SPARSE INDEX (one entry per block - file MUST be sorted):
  Index Entries           DATA FILE (SORTED by empid)
  +-----------+         +----------------------------------+
  | 10 | ptr1 |-------->| BLOCK1: {empid=10},{empid=20}    |
  | 30 | ptr2 |-------->| BLOCK2: {empid=30},{empid=40}    |
  | 50 | ptr3 |-------->| BLOCK3: {empid=50},{empid=60}    |
  +-----------+         +----------------------------------+
  6 records in 3 blocks = only 3 index entries

To find empid=40: index entry for 30 <= 40, go to BLOCK2, scan within
block.
```

Note: Sparse index is much smaller than dense. Use dense when file unsorted or when direct record access is critical. Use sparse when file is sorted and index size/maintenance overhead matters.

Multilevel Index:

When the index itself is too large to fit in memory (requires many disk I/Os to search), a second-level sparse index is built on top of the first-level index. This creates a multilevel index structure.

Multilevel Index Structure

```
  Level 2 (outer sparse index on level 1):
  +------+   +------+   +------+
  | 100  |   | 400  |   | 700  |  <- outer index entries
  +------+   +------+   +------+
      |           |           |
  Level 1 (inner dense/sparse index):
  +--+--+--+  +--+--+--+  +--+--+--+
  |100|200|300||400|500|600||700|800|900|  <- inner index entries
  +--+--+--+  +--+--+--+  +--+--+--+
       |            |            |
  Actual data file blocks
```

```
  Search: Read outer index (1 I/O) -> inner index block (1 I/O) -> data
block (1 I/O) = 3 I/Os total
```

```
  This is exactly how B+ tree works! Each level is a sparse index on the
level below.
```

Note: Multilevel indexing is the conceptual foundation of the B+ Tree. B+ tree generalizes multilevel indexing with dynamic insertion/deletion and balance guarantees.

SECTION 4: B+ Tree Index Files

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q8. What is a B+ Tree? Explain its structure with internal and leaf nodes. State all properties of B+ tree. Explain Search, Insertion (with split), and Deletion (with merge/redistribute) operations with examples.

[2+4+4]
2082 Kartik 2079 Chaitra 2077 Chaitra

Answer: B+ Tree Index Files

B+ Tree: A B+ Tree is a self-balancing, ordered tree used as a multilevel index in DBMS. ALL data pointers (record pointers) are stored ONLY in LEAF nodes. Internal nodes store only keys as routing guides. All leaf nodes are linked in a sorted doubly-linked list enabling efficient range queries.

Properties of B+ Tree of Order n (max n pointers per node):

* INTERNAL NODES: Hold at most n pointers and n-1 keys. Hold at least ceil(n/2) pointers (except root).
* LEAF NODES: Hold at most n-1 key-pointer pairs. Hold at least ceil((n-1)/2) entries. One pointer links to next leaf.
* ROOT: Has at least 2 children (unless it is also a leaf).
* ALL LEAVES are at the SAME LEVEL - tree is always perfectly balanced.
* Leaf nodes form a SORTED LINKED LIST - enables efficient range scans.
* Keys in internal nodes are COPIES of keys in leaves (routing purposes only).

Node Structure:

Node Type	Structure	Contents
Internal Node	[P1 | K1 | P2 | K2 | P3 | ... | Pn]	Pi = pointer to child subtree. Ki = search key. All keys in P1 subtree < K1. K1 <= keys in P2 subtree < K2. Etc.
Leaf Node	[K1|R1 | K2|R2 | ... | Kn-1|Rn-1 | Next->]	Ki = search key value. Ri = pointer to data record with key Ki. Last pointer links to next leaf node for range scans.

B+ Tree Example (Order n=4, max 3 keys per node)

```
                    ROOT (Internal Node)
            +-----------------------------+
            |  P1  |  30  |  P2  |  60  |  P3  |
            +-----------------------------+
           /               |               \
      (keys < 30)   (30 <= keys < 60)   (keys >= 60)
           |                 |                 |
    +-------------+  +--------------+  +----------+
    |10->R1|20->R2|->|30->R3|40->R4|->|60->R5|70->R6|->NULL
      LEAF 1    LEAF 2          LEAF 3

   Internal node: [30, 60] - routing keys only (no data pointers)
   Leaf nodes: all data pointers + linked list (LEAF1->LEAF2->LEAF3)
   Range query "30 to 60": go to LEAF2, scan list until key > 60.
```

Note: All record pointers are in leaves only. Internal nodes guide the search path.

B+ Tree Search:

* Step 1: Start at root. Compare search key K with keys in current node.
* Step 2: In internal node: if K < K1 follow P1; if K1<=K<K2 follow P2; else follow next pointer.
* Step 3: Repeat traversal until a leaf node is reached.
* Step 4: At leaf: search linearly for K. If found, follow record pointer. If not found, key does not exist.
* Cost: O(log_n N) disk I/Os where N = total records. Typically 3-4 I/Os for millions of records.

B+ Tree Insertion (with split):

* Step 1: Search for the correct LEAF NODE for the new key.
* Step 2a: If leaf has space (< n-1 keys): insert in sorted position. DONE.
* Step 2b: If leaf is FULL: SPLIT the leaf node:
* (a) Create a new leaf. Move the upper ceil(n/2) entries to the new leaf.
* (b) COPY-UP the smallest key of the new leaf to the parent internal node.
* (c) Link new leaf into the leaf linked list.
* (d) If parent internal node is also full: PUSH-UP the middle key to grandparent and split parent.
* (e) If root splits: create a new root. Tree height increases by 1.

B+ Tree Insertion Example (Order n=3, max 2 keys/leaf)

```
Insert 10, 20, 30, 40, 50 into empty B+ tree:

Insert 10: LEAF=[10]
Insert 20: LEAF=[10|20]  (leaf now full)
Insert 30: LEAF full -> SPLIT
```

```
  LEAF1=[10|20], LEAF2=[30]  Copy-up 30 to root
  ROOT=[30]  LEAF1=[10|20]->LEAF2=[30]

Insert 40: goes to LEAF2 -> LEAF2=[30|40] (full)
Insert 50: LEAF2 full -> SPLIT
  LEAF2=[30|40], LEAF3=[50]  Copy-up 50 to root
  ROOT=[30|50]  LEAF1=[10|20]->LEAF2=[30|40]->LEAF3=[50]

Final B+ Tree:
       ROOT: [30 | 50]
       /         |      \
 [10|20]     [30|40]   [50]
  LEAF1  -->  LEAF2 --> LEAF3 --> NULL
```

Note: Copy-up for LEAF split (key copied to parent AND stays in leaf). Push-up for INTERNAL node split (key moves to parent, removed from child). This is a critical distinction.

B+ Tree Deletion (with merge/redistribute):

* Step 1: Find the leaf node containing the key. Remove the key-pointer pair from leaf.
* Step 2a: If leaf has >= ceil((n-1)/2) entries after deletion: DONE (no underflow).
* Step 2b: If UNDERFLOW (too few entries):
* (a) REDISTRIBUTE: try to borrow an entry from a sibling leaf (left or right). Update parent key.
* (b) If sibling cannot lend (also at minimum): MERGE leaf with sibling. Remove separating key from parent.
* (c) If parent underflows after key removal: propagate merge/redistribute upward recursively.
* (d) If root has only 1 child after merge: remove root. Tree height decreases by 1.

Most Repeated: "Explain B+ Tree with insertion and deletion with examples" is the single most repeated question in this chapter, appearing almost every exam. Always show: node structure, split (copy-up vs push-up distinction), and delete with merge/redistribute.

Q9. How does a B-Tree differ from a B+ Tree? Draw the structure of each and compare their advantages, disadvantages, and use cases.

[2+4+2]

2079 Ashwin Back 2078 Chaitra 2073 Bhadra

Answer: B-Tree vs B+ Tree

Feature	B-Tree	B+ Tree
Data Pointer Location	In EVERY node (internal AND leaf). Each node has both keys and data pointers.	ONLY in LEAF nodes. Internal nodes contain keys only (routing).

Key Duplication	Each key stored EXACTLY ONCE - at the node where its data pointer is.	Keys in internal nodes are DUPLICATED in leaf nodes.
Leaf Linking	Leaf nodes NOT linked together.	All leaf nodes linked in sorted DOUBLY-LINKED LIST.
Range Queries	POOR - must traverse tree for each key in range (in-order traversal).	EXCELLENT - find start in tree, scan leaf linked list.
Search (single key)	Can find key in internal node (no need to reach leaf) - sometimes faster.	Always traverses to leaf. Guaranteed log n levels.
Tree Height	Taller - internal nodes hold fewer keys (space taken by data pointers).	Shorter - internal nodes hold more keys (no data pointers) = higher fan-out.
Fan-out (keys per node)	Lower (data pointers consume node space).	Higher (only keys in internal nodes) = shorter tree = fewer I/Os.
Preferred In	In-memory indexes, file systems (NTFS, HFS+), small trees.	All major RDBMS (MySQL InnoDB, PostgreSQL, Oracle, SQL Server) - range queries common.

B-Tree vs B+ Tree - Structural Difference

```
B-TREE (data pointers in EVERY node):
                [20,D20 | 40,D40]   <- Internal has data ptrs
               /                 \
    [10,D10 | 15,D15]        [50,D50 | 60,D60]

  - Finding key=40: STOP at root (found D40). No need to go to leaf.
  - Range query 20 to 50: must traverse tree multiple times.

B+ TREE (data only in leaves):
                [20 | 40]           <- Internal: keys only (routing)
               /         \
    [10->R10|15->R15]->   [20->R20|40->R40|50->R50|60->R60]->NULL
         LEAF1         ->                LEAF2

  - Finding key=40: traverse to leaf, get R40 (data pointer).
  - Range query 20 to 50: go to leaf with 20, scan linked list to 50.
  - Internal node: no data pointers -> more keys fit -> lower height ->
fewer I/Os.
```

Note: B+ Tree is preferred for DBMS because: (1) range queries via leaf linked list, (2) higher fan-out means shorter tree, (3) more consistent performance (always reaches leaf).

SECTION 5: Hashing - Static and Extendible Dynamic Hashing

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q10. What is Hashing in DBMS? Explain Static Hashing with hash functions, buckets, overflow chaining, and bucket overflow. What are the limitations of static hashing?

[2+5+1]
2081 Chaitra 2079 Chaitra 2075 Bhadra

Answer: Static Hashing

Hashing: Hashing provides direct O(1) average access to records based on a search key. A hash function h(K) maps key K to a bucket number (0 to N-1). Records with keys mapping to the same bucket are stored together. Ideal for exact-match (equality) queries.

Key Concepts in Static Hashing:

Concept	Definition
Hash Function h(K)	Maps search key K to a bucket number b = h(K) in range [0, N-1]. A good hash function distributes keys UNIFORMLY across N buckets. Common: h(K) = K mod N.
Bucket	A unit of storage (typically one disk block) holding records whose keys hash to the same value. Identified by bucket number 0 to N-1.
Bucket Address Table	An array of size N where entry i holds the disk address (pointer) of bucket i. The DBMS uses this to locate any bucket in one lookup.
Collision	Two different keys K1 != K2 hash to same bucket: h(K1) = h(K2). Inevitable and must be handled gracefully.
Overflow Chaining	When a bucket fills up, a linked OVERFLOW BUCKET is allocated. Records that hash to a full bucket are stored in the overflow chain.
Bucket Overflow	Occurs when a primary bucket is full and a new record must be stored. Triggers creation of an overflow bucket.

Static Hashing Structure - Example

```
Hash Function: h(K) = K mod 4   (4 buckets: 0,1,2,3)

Bucket Address Table:
  [0] --> BLOCK: {K=4, K=8, K=12} --> OVERFLOW: {K=16}
  [1] --> BLOCK: {K=1, K=5, K=9}
  [2] --> BLOCK: {K=2, K=6, K=10}
```

```
  [3] --> BLOCK: {K=3, K=7, K=11}

Operations:
  Insert K=15: h(15)=15 mod 4=3 -> Bucket[3]: {K=3,K=7,K=11,K=15}
  Insert K=16: h(16)=16 mod 4=0 -> Bucket[0] full -> create overflow
  Search K=9:  h(9)=9 mod 4=1  -> Bucket[1]: found K=9 directly
  Delete K=8:  h(8)=8 mod 4=0  -> Bucket[0]: remove K=8
```

Note: Average search: O(1) - compute hash, go to bucket, scan bucket. Worst case: O(n) if all keys hash to same bucket (bad hash function or many overflows).

Limitations of Static Hashing:

Limitation	Explanation
Fixed number of buckets (N)	N is chosen at creation time. Database growth causes bucket overload and long overflow chains - performance degrades to O(n).
Over-allocation	Choosing N very large wastes disk space with empty buckets.
Cannot resize without full rebuild	Changing N requires rehashing ALL records. Extremely expensive operation.
Skew problem	Non-uniform hash function creates hot buckets (some very full, others empty).
No range query support	Hashing destroys key order - cannot answer range queries (a <= K <= b).

Q11. What is Extendible (Dynamic) Hashing? Explain with Global Depth, Local Depth, and Directory. Show insertion with directory doubling using a step-by-step example. How does it overcome static hashing limitations?

[2+6]
2080 Ashwin 2078
Baishakh 2074 Bhadra

Answer: Extendible Hashing

Extendible Hashing: Extendible Hashing is a dynamic hashing technique where the hash structure grows and shrinks as records are inserted or deleted. It uses a DIRECTORY of pointers to buckets. The directory DOUBLES when needed instead of rebuilding the entire file. Only the overflowing bucket splits.

Key Concepts:

Concept	Definition
Global Depth (i)	Number of hash bits used to index the DIRECTORY. Directory has 2^i entries. If i=2, directory has 4 entries indexed by 2-bit prefixes: 00, 01, 10, 11.

Local Depth (j)	Each bucket has its own local depth j = number of hash bits that distinguish keys in that bucket. Always j <= i.
Directory	Array of 2^i pointers to buckets. Multiple directory entries may point to the SAME bucket (when j < i).
Bucket Split Rule	When a bucket with local depth j overflows: IF j < i: split bucket only, no directory change. IF j = i: double directory (i = i+1), then split bucket.

Extendible Hashing - Step-by-Step Insertion Example

```
Hash: use last i bits. Bucket capacity = 2 records. Initial: i=1.

State 0: i=1, Directory: [0->BktA, 1->BktB]
         BktA(j=1):[] BktB(j=1):[]

Insert 2 (binary ...010, last 1 bit=0) -> BktA
Insert 3 (binary ...011, last 1 bit=1) -> BktB
Insert 4 (binary ...100, last 1 bit=0) -> BktA
  State: i=1, [0->BktA:{2,4}, 1->BktB:{3}]  (BktA now FULL)

Insert 5 (last bit=1) -> BktB: BktB={3,5} (FULL now too)

Insert 6 (last bit=0) -> BktA FULL!
  BktA local depth j=1 = global depth i=1 => DOUBLE DIRECTORY
  i becomes 2. Directory expands: [00, 01, 10, 11] (4 entries)
  Split BktA by last 2 bits:
    BktA1 (last 2 bits=00): records with ...00 -> {4}
    BktA2 (last 2 bits=10): records with ...10 -> {2, 6}
  BktB still has j=1, so both 01 and 11 point to BktB

Final state: i=2
  Dir[00] -> BktA1(j=2): {4}
  Dir[01] -> BktB(j=1):  {3,5}
  Dir[10] -> BktA2(j=2): {2,6}
  Dir[11] -> BktB(j=1):  {3,5}  <- same bucket as Dir[01]

Search K=6: h(6)=110, last 2 bits=10 -> Dir[10] -> BktA2 -> found!
Search K=3: h(3)=011, last 2 bits=11 -> Dir[11] -> BktB -> found!
```

Note: Key: when j<i, split bucket only (no directory doubling). When j=i, double directory then split. Multiple directory entries can point to same bucket (when j<i for that bucket).

Feature	Static Hashing	Extendible Hashing
Bucket Count	Fixed N at creation. Cannot grow.	Grows dynamically via bucket splits and directory doubling.
Overflow	Overflow chains degrade to O(n).	No overflow chains - bucket splits prevent overflow.
Reorganization	Full file rebuild to resize (rehash all records).	Only directory doubles or bucket splits. Localized change.
Directory	No directory (direct address table).	Directory of 2^i pointers (doubles when needed).
Search Cost	O(1) best, O(n) with long overflow chains.	O(1) average - one directory lookup + bucket scan.
Best For	Static databases with known size.	Dynamic databases with unpredictable growth/shrinkage.

SECTION 6: Query Processing and Optimization

Exam Ref: 2082 Kartik, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2073 Bhadra, 2072 Ashwin

Q12. What is Query Processing? Explain the three steps: parsing, optimization, and evaluation. What is a query execution plan? What is Query Optimization and how does the optimizer work?

[2+4+2]
2081 Ashwin 2079 Chaitra 2075 Bhadra

Answer: Query Processing

Query Processing: Query processing refers to activities involved in extracting data from a database in response to a SQL query. It transforms a high-level SQL query (declarative - what) into a low-level sequence of operations (procedural - how) for actual execution.

Three Steps of Query Processing:

Step	Phase	Input	Output	Key Activity
1	Parsing and Translation	SQL query string	Internal relational algebra expression (parse tree)	Syntax check. Table/column name validation using data dictionary. Convert SQL to equivalent relational algebra expression.
2	Optimization	Relational algebra expression	Optimal query execution plan (QEP)	Generate all equivalent plans using transformation rules. Estimate cost of each plan. Select the lowest-cost plan.
3	Evaluation (Execution)	Query execution plan	Query result (tuples)	Execute the chosen plan: read data from files/indexes, apply operators (select, join, sort, project), return results to user.

Query Processing Pipeline

```
          SQL Query (User Input)
                |
     +----------v----------+
     |  PARSER & TRANSLATOR |  -> Check syntax, validate table/column names
     +--------------------+  -> Translate SQL to Relational Algebra
 expression
                |
          Relational Algebra Expression
          (e.g., sigma_salary>50K(Employee join Department))
                |
     +----------v----------+
     |   QUERY OPTIMIZER   |  -> Generate equivalent plans
     +--------------------+  -> Estimate cost using statistics
                |            -> Choose lowest-cost plan
          Query Execution Plan (QEP)
          (e.g., Index scan on salary index, then nested-loop join)
                |
     +----------v----------+
     |  EVALUATION ENGINE  |  -> Execute QEP step by step
     +--------------------+  -> Access files, indexes, apply operators
                |
          Query Result (Output to User)
```

Note: The optimizer is the heart of query processing. It uses statistics from the data dictionary to estimate I/O costs and pick the best execution strategy.

Query Cost Measures:

Cost Factor	Description	Dominant?
Disk I/O (Block Transfers)	Number of blocks read from and written to disk. Each block transfer is slow (~10ms). Measured in number of block transfers (bt).	YES - dominant cost
CPU Cost	Number of in-memory operations (comparisons, arithmetic, hashing).	No - usually ignored
Memory (Buffer)	Number of buffer blocks available. More memory = more blocks cached = fewer disk reads.	Indirect - affects I/O
Network Cost	Data transfer cost in distributed DBMS.	Only in distributed

Algebraic Equivalence Rules (for plan generation):

* Selection pushdown: sigma_(c1 AND c2)(R) = sigma_c1(sigma_c2(R)) - split and push selections down.
* Selection before join: sigma_c(R join S) = sigma_c(R) join S - reduces join input size.
* Projection pushdown: project early to eliminate unused columns before joins.
* Join reordering: (R join S) join T = R join (S join T) - change join order for efficiency.
* Join commutativity: R join S = S join R - swap sides of join.

Q13. What algorithms are used for computing SELECT and JOIN in query processing? Explain Nested Loop Join, Sort-Merge Join, and Hash Join with their costs.

[2+6]
2080 Ashwin 2078
Chaitra 2076 Baisakh

Answer: Query Evaluation Algorithms

Algorithms for SELECT Operation (sigma):

Algorithm	How It Works	Cost	Best When
Linear (Full) Scan	Read every block of the relation from start to end. Examine each record.	b_r blocks (all blocks)	No index exists; relation is small.
Binary Search	If relation sorted on attribute, use binary search to find matching block.	ceil(log2(b_r)) blocks	File sorted on search attribute; no index.
Primary B+ Tree Index	Traverse B+ tree to leaf. Get record pointer. One record fetched.	Height_i + 1 blocks	Primary key equality query.
Secondary B+ Tree Index	Traverse B+ tree to leaf. Each matching entry may be in different block.	Height_i + matches blocks	Non-primary attribute equality; few matches.
Primary Index - Range	Traverse tree to start of range, scan leaf list (data clustered).	Height_i + blocks in range	Range query on clustered index.

Algorithms for JOIN Operation:

Algorithm	How It Works	Cost (Block Transfers)	Best When
Nested-Loop Join (Tuple)	For each TUPLE in outer r: scan ALL of inner s for matching tuples.	n_r x b_s + b_r	Tiny inner relation or very small join.

Block Nested-Loop Join	For each BLOCK of outer r: read all blocks of inner s.	b_r x b_s + b_r (limited memory)	Reduces I/Os vs tuple nested-loop. Most common fallback.
Index Nested-Loop Join	For each tuple in outer r: use INDEX on s to find matching tuples.	b_r + n_r x c (c=index cost)	Index exists on join attribute of inner relation.
Sort-Merge Join	SORT both relations on join attribute. Scan and MERGE matching tuples.	Sort(r) + Sort(s) + b_r + b_s	Relations already sorted or affordable sort cost.
Hash Join	HASH both relations on join attribute into buckets. Join matching buckets.	3(b_r + b_s) typical	Large unsorted relations; no index; enough memory.

Exam Tip: For exam: Hash join is best for large unsorted relations. Index nested-loop is best when index exists on inner relation. Sort-merge is best when both sides already sorted or query has ORDER BY. Always compare costs using b_r and b_s notation.

SECTION 7: Data Dictionary and Index Comparison Summary

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2079 Chaitra, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra

Q14. What is a Data Dictionary (System Catalog) in DBMS? What information does it store? How does the DBMS use the data dictionary?

[2+4+2]
2081 Chaitra 2079 Chaitra 2077 Chaitra

Answer: Data Dictionary (System Catalog)

Data Dictionary: The Data Dictionary (also called System Catalog or Metadata Repository) is a special database maintained by the DBMS that stores METADATA - data about the structure of the database itself. It stores schemas, not actual user data. Every DBMS operation consults the data dictionary.

Information Stored in the Data Dictionary:

Category	Information Stored	Example
Relation Information	Table name, attribute names, data types, sizes, constraints (PK, FK, NOT NULL, UNIQUE, CHECK), default values.	Employee: empid INT PK, ename VARCHAR(100) NOT NULL, salary DECIMAL(10,2)

Index Information	Index name, table name, indexed columns, index type (B+ tree, hash), clustered/non-clustered.	idx_emp_salary: B+ tree on Employee(salary), non-clustered.
View Definitions	View name, SQL definition, columns exposed to users.	EmpView = SELECT empid,ename FROM Employee WHERE dept=CS
User and Authorization	User accounts, roles, privileges (SELECT/INSERT/UPDATE/DELETE) on tables and views.	User Ram: SELECT on Employee, GRANT INSERT to Sita.
Statistical Information	Tuple counts, distinct values per attribute, histograms, min/max values. Used by optimizer.	Employee: 5000 tuples, 20 distinct dept_ids, salary 20K-200K.
Storage Information	Physical file names, block sizes, disk locations of each relation.	Employee stored in /data/emp.dat, block size 8KB, 625 blocks.

How DBMS Uses the Data Dictionary:

* Parser: Verifies that table and column names in SQL queries actually exist.
* Semantic Checker: Validates data types and access privileges before execution.
* Query Optimizer: Uses statistics (tuple counts, distinct values) to estimate costs and choose best plan.
* Transaction Manager: Enforces integrity constraints (PK, FK) during updates.
* DBA: Administrators query the data dictionary to analyze schema, performance, and storage.

Q15. Compare B+ Tree index and Hash index. When would you choose each? Also compare ordered (B+ tree) vs unordered (hash) indexes on: equality queries, range queries, sorting, and insertion cost.

[2+4+2]
2080 Ashwin 2078
Baishakh 2073 Bhadra

Answer: B+ Tree Index vs Hash Index

Feature	B+ Tree Index	Hash Index
Data Structure	Balanced tree. Leaves hold key-ptr pairs in sorted order; linked list.	Hash table. h(K) maps key to bucket. Records in buckets.
Equality Query K=v	O(log n) - traverse root to leaf.	O(1) average - compute h(v), go to bucket.
Range Query a<=K<=b	EXCELLENT - find lower bound, scan leaf linked list.	NOT SUPPORTED - hashing destroys key order.
ORDER BY / Sorting	Supported - in-order leaf traversal gives sorted result.	Not supported - no inherent order.

Prefix Match K LIKE A%	Supported - ordered structure enables prefix search.	Not supported.
Worst-Case Search	O(log n) - guaranteed (always balanced).	O(n) - if all keys in same bucket (bad hash or heavy collisions).
Insert Cost	O(log n) with possible node splits.	O(1) average with possible rehashing or overflow.
Space Overhead	Moderate - tree nodes + leaf list pointers.	Low (static) to moderate (extendible with directory).
Use In DBMS	Default index in MySQL InnoDB, PostgreSQL, Oracle, SQL Server.	Hash join operations, equality-only lookups, some in-memory indexes.

Exam Tip: Choose B+ Tree: when range queries, ORDER BY, or multi-condition queries needed (most cases). Choose Hash: only for equality-only lookups where range queries are never needed and maximum insert/lookup speed is desired.

QUICK REVISION: Chapter 6 - Key Points Summary

Topic	Key Points	Expected Marks
Storage Hierarchy	Cache(volatile,fast) -> RAM -> SSD -> HDD(10ms) -> Tape(cheapest). Primary=volatile. Secondary+Tertiary=non-volatile. Block=unit of I/O between disk and memory.	2-4
Disk Structure	Platter, Track, Sector(512B/4KB), Block(4-16KB), Cylinder, Disk Arm, R/W Head. Access = Seek Time(dominant ~5-10ms) + Rotational Latency(~4ms) + Transfer Time(~0.04ms).	2-4
RAID	0=striping(no fault tolerance). 1=mirroring(50% efficiency). 5=distributed parity((N-1)/N). 6=double parity. 10=stripe+mirror(50%, best performance+reliability).	2-4
Buffer Manager	Buffer pool caches hot disk blocks in RAM. Pin count: pinned page cannot be evicted. Dirty page must be written to disk before eviction. LRU/MRU/Clock replacement policies.	2-4
File Organization	Heap(unsorted,fast insert,slow search). Sequential(sorted,binary search,slow insert). Hash(O(1) exact match,no range). Clustered(related rows together).	2-4

Dense vs Sparse Index	Dense: one entry per RECORD, any file. Sparse: one entry per BLOCK, requires sorted file. Dense=fast,large. Sparse=small,needs scan within block. Sparse=basis for multilevel/B+tree.	3-6
B+ Tree	ALL data ptrs in leaves. Leaves linked. Order n: internal <=n ptrs, leaf <=n-1 entries, min=ceil/2. Insert: COPY-UP for leaf split, PUSH-UP for internal split. Delete: redistribute or merge. O(log n).	4-8
B-Tree vs B+ Tree	B-Tree: data in ALL nodes, no leaf link, lower fan-out, good for in-memory/filesystems. B+ Tree: data only in leaves, leaves linked, higher fan-out, better range queries, used in all RDBMS.	2-4
Static Hashing	h(K)=K mod N -> bucket. Overflow chaining when bucket full. Limits: fixed N, overflow degrades to O(n), full rebuild needed to resize.	2-4
Extendible Hashing	Global depth i (directory size=2^i). Local depth j per bucket. Split: if j<i -> bucket split only. If j=i -> double directory then split. No overflow chains. No full rebuild. O(1) avg.	4-8
Query Processing	Parse (SQL -> RA expression) -> Optimize (choose best plan by estimated cost) -> Evaluate (execute plan). Cost = disk block transfers. Optimizer uses statistics from data dictionary.	2-4
Join Algorithms	Nested-loop O(nr*bs). Block nested-loop O(br*bs). Index nested-loop O(br+nr*c). Sort-merge O(sort+br+bs). Hash-join O(3(br+bs)). Choose based on: index availability, relation sizes, sort order.	2-4
Data Dictionary	Stores metadata: schemas, indexes, views, users, statistics, constraints. Used by parser(validate names), optimizer(statistics), transaction manager(enforce constraints).	1-3